

Exercises

Numerical experiments

9.30 *Gradient and Newton methods.* Consider the unconstrained problem

$$\text{minimize } f(x) = -\sum_{i=1}^m \log(1 - a_i^T x) - \sum_{i=1}^n \log(1 - x_i^2),$$

with variable $x \in \mathbf{R}^n$, and $\text{dom } f = \{x \mid a_i^T x < 1, i = 1, \dots, m, |x_i| < 1, i = 1, \dots, n\}$. This is the problem of computing the analytic center of the set of linear inequalities

$$a_i^T x \leq 1, \quad i = 1, \dots, m, \quad |x_i| \leq 1, \quad i = 1, \dots, n.$$

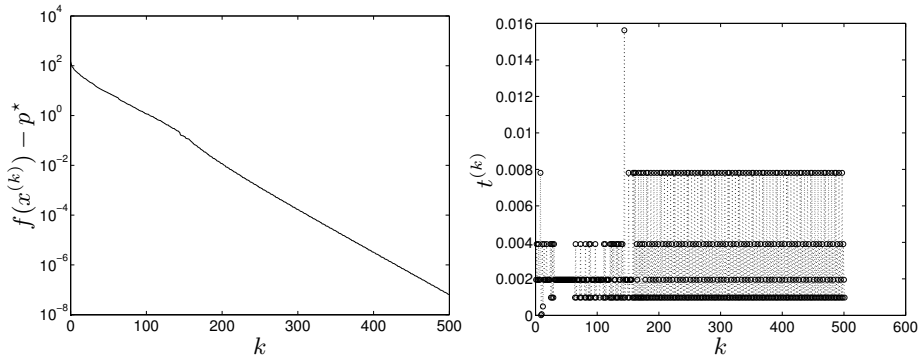
Note that we can choose $x^{(0)} = 0$ as our initial point. You can generate instances of this problem by choosing a_i from some distribution on $\mathbf{R}^n$.

- Use the gradient method to solve the problem, using reasonable choices for the backtracking parameters, and a stopping criterion of the form $\|\nabla f(x)\|_2 \leq \eta$. Plot the objective function and step length versus iteration number. (Once you have determined p^* to high accuracy, you can also plot $f - p^*$ versus iteration.) Experiment with the backtracking parameters α and β to see their effect on the total number of iterations required. Carry these experiments out for several instances of the problem, of different sizes.
- Repeat using Newton's method, with stopping criterion based on the Newton decrement λ^2 . Look for quadratic convergence. You do not have to use an efficient method to compute the Newton step, as in exercise 9.27; you can use a general purpose dense solver, although it is better to use one that is based on a Cholesky factorization.

Hint. Use the chain rule to find expressions for $\nabla f(x)$ and $\nabla^2 f(x)$.

Solution.

- Gradient method.* The figures show the function values and step lengths versus iteration number for an example with $m = 200$, $n = 100$. We used $\alpha = 0.01$, $\beta = 0.5$, and exit condition $\|\nabla f(x^{(k)})\|_2 \leq 10^{-3}$.



The following is a Matlab implementation.

```
ALPHA = 0.01;
BETA = 0.5;
MAXITERS = 1000;
GRADTOL = 1e-3;

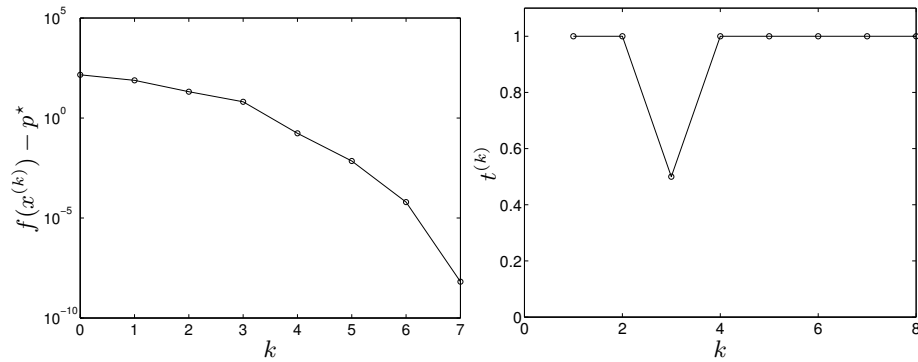
x = zeros(n,1);
for iter = 1:MAXITERS
    val = -sum(log(1-A*x)) - sum(log(1+x)) - sum(log(1-x));
    grad = A'*(1./(1-A*x)) - 1./(1+x) + 1./(1-x);
    if norm(grad) < GRADTOL, break; end;
```

```

v = -grad;
fprime = grad'*v;
t = 1; while ((max(A*(x+t*v)) >= 1) | (max(abs(x+t*v)) >= 1)),
    t = BETA*t;
end;
while ( -sum(log(1-A*(x+t*v))) - sum(log(1-(x+t*v).^2)) > ...
    val + ALPHA*t*fprime )
    t = BETA*t;
end;
x = x+t*v;
end;

```

- (b) *Newton method.* The figures show the function values and step lengths versus iteration number for the same example. We used $\alpha = 0.01$, $\beta = 0.5$, and exit condition $\lambda(x^{(k)})^2 \leq 10^{-8}$.



The following is a Matlab implementation.

```

ALPHA = 0.01;
BETA = 0.5;
MAXITERS = 1000;
NTTOL = 1e-8;

x = zeros(n,1);
for iter = 1:MAXITERS
    val = -sum(log(1-A*x)) - sum(log(1+x)) - sum(log(1-x));
    d = 1./(1-A*x);
    grad = A'*d - 1./(1+x) + 1./(1-x);
    hess = A'*diag(d.^2)*A + diag(1./(1+x).^2 + 1./(1-x).^2);
    v = -hess\grad;
    fprime = grad'*v;
    if abs(fprime) < NTTOL, break; end;
    t = 1; while ((max(A*(x+t*v)) >= 1) | (max(abs(x+t*v)) >= 1)),
        t = BETA*t;
    end;
    while ( -sum(log(1-A*(x+t*v))) - sum(log(1-(x+t*v).^2)) > ...
        val + ALPHA*t*fprime )
        t = BETA*t;
    end;
    x = x+t*v;
end;

```

- 9.31** *Some approximate Newton methods.* The cost of Newton's method is dominated by the cost of evaluating the Hessian $\nabla^2 f(x)$ and the cost of solving the Newton system. For large